

Java Backend Interview Prep - JVM and Java 8

This document answers the JVM and Java 8 topics listed in the source interview sheet.[file:1]

JVM basics

1) What is JVM? Difference between JVM, JRE, and JDK

JVM is the runtime engine that executes Java bytecode. It provides platform independence because the same `.class` files can run on any OS that has a compatible JVM.[file:1]

- **JVM:** Executes bytecode, manages memory, performs GC, JIT compilation, and thread scheduling support.
- **JRE:** JVM plus standard libraries needed to run Java applications.
- **JDK:** JRE plus development tools such as `javac`, `javadoc`, `jar`, and debuggers.

Interview line: *JDK is for development, JRE is for running, JVM is the execution engine inside the JRE.*

2) JVM architecture and runtime data areas

Main components:[file:1]

- **Class Loader Subsystem:** Loads `.class` files into memory and links them.
- **Runtime Data Areas:** Heap, stacks, method area/metaspace, PC register, native method stack, code cache.
- **Execution Engine:** Interpreter + JIT compiler + garbage collector.

Important memory areas:

- **Heap:** Shared across threads; stores objects and arrays.
- **Java Stack:** Per-thread; stores stack frames, local variables, method calls.
- **Metaspace:** Class metadata, loaded class definitions.
- **Code Cache:** Stores JIT-compiled native code.
- **PC Register:** Per-thread pointer to current bytecode instruction.
- **Native Method Stack:** Used by JNI/native calls.

3) Thread-specific runtime data areas

Per-thread areas are:[file:1]

- Java Stack
- PC Register
- Native Method Stack

Impact of thread creation:

- Each new thread allocates its own stack memory.
- Too many threads can cause high memory usage and context switching.
- Large `-Xss` values reduce how many threads the process can create.

4) Java Memory Model (JMM)

JMM defines how threads interact through memory and what visibility, ordering, and atomicity guarantees Java provides.[file:1]

Core guarantees:

- **Visibility:** A write by one thread becomes visible to another under defined synchronization rules.
- **Ordering:** Compiler/CPU cannot freely reorder everything when synchronization constructs are present.
- **Atomicity:** Some operations are atomic, but compound operations like `count++` are not.

5) Happens-before relationship

A happens-before rule guarantees that writes before one action are visible to reads after another action.

Common examples:

- Unlock on a monitor happens-before the next lock on the same monitor.
- Write to a `volatile` field happens-before a later read of that field.
- `Thread.start()` happens-before actions in the started thread.
- All actions in a thread happen-before another thread successfully returns from `join()`.

6) Volatile and atomic operations

`volatile` guarantees visibility and ordering for a variable, but it does not make compound operations atomic.

```
class FlagExample {
    private volatile boolean shutdown = false;

    public void stop() {
        shutdown = true;
    }

    public void runLoop() {
        while (!shutdown) {
            // do work
        }
    }
}
```

```
}  
}
```

For atomic compound operations, use `AtomicInteger`, `LongAdder`, or synchronization.

```
import java.util.concurrent.atomic.AtomicInteger;  
  
class Counter {  
    private final AtomicInteger count = new AtomicInteger();  
  
    public int increment() {  
        return count.incrementAndGet();  
    }  
}
```

7) Frequent Full GC - analysis and tuning approach

A strong interview answer should be structured:[file:1]

1. Check GC logs: frequency, pause time, old-gen occupancy, allocation rate.
2. Capture JFR: allocation hotspots, lock contention, safepoints, IO stalls.
3. Check heap dump if memory leak is suspected.
4. Review object churn: large temporary collections, excessive string creation, cache misuse.
5. Inspect Metaspace/classloader leaks in plugin or hot-reload environments.
6. Verify heap sizing: undersized heap causes frequent collection, oversized heap can increase pause times.
7. Choose/tune collector based on workload: G1, ZGC, Shenandoah, Parallel GC.

Common resolutions:

- Reduce object allocation rate.
- Fix memory leaks.
- Right-size `-Xms`, `-Xmx`.
- Tune young/old generation behavior if needed.
- Limit large in-memory caches.
- Upgrade to a newer JDK if GC behavior improves for the workload.

8) Useful JVM arguments

Examples often discussed in interviews:[file:1]

```
-Xms2g -Xmx2g  
-Xss512k  
-XX:+UseG1GC  
-XX:MaxGCPauseMillis=200  
-XX:+HeapDumpOnOutOfMemoryError
```

```
-XX:HeapDumpPath=/tmp/heap.hprof
-Xlog:gc*:file=gc.log:time,uptime,level,tags
-XX:MaxMetaspaceSize=512m
-XX:+UseStringDeduplication
```

Do not present tuning flags as universal best practices; they are workload dependent.

9) Metaspace vs PermGen

- **PermGen** existed before Java 8 and had fixed-size tuning problems.
- **Metaspace** replaced PermGen in Java 8 and stores class metadata in native memory.
- Metaspace usually grows dynamically until a limit is reached or native memory is exhausted.[file:1]

Java 8 concepts

10) Functional interface

A functional interface has exactly one abstract method. It was added to support lambda expressions and method references.[file:1]

@FunctionalInterface is optional, but recommended because it gives compile-time validation.

```
@FunctionalInterface
interface Adder {
    int add(int a, int b);
}
```

11) Predicate vs Function vs Supplier

- **Predicate<T>**: takes input, returns boolean.
- **Function<T, R>**: takes input, returns transformed output.
- **Supplier<T>**: takes no input, returns output.

```
Predicate<String> isLong = s -> s.length() > 5;
Function<String, Integer> length = String::length;
Supplier<Long> now = System::currentTimeMillis;
```

12) Custom FI example

```
@FunctionalInterface
interface MathOp {
    int apply(int a, int b);
}

public class Demo {
    public static void main(String[] args) {
```

```

        MathOp sum = (a, b) -> a + b;
        System.out.println(sum.apply(10, 20));
    }
}

```

13) Default methods

Default methods allow interfaces to add new behavior without breaking existing implementations.

```

interface Vehicle {
    default void start() {
        System.out.println("Vehicle starting");
    }
}

```

14) Method references

Method references are a shorter form of lambdas when an existing method already matches the target functional interface.

Types:

- `ClassName::staticMethod`
- `objectRef::instanceMethod`
- `ClassName::instanceMethod`
- `ClassName::new`

Sort ignoring case:

```

List<String> names = Arrays.asList("bob", "Alice", "charlie");
names.sort(String::compareToIgnoreCase);

```

15) Lambdas vs anonymous inner classes

Key differences:[file:1]

- Lambda does not create a new named scope for `this`; `this` refers to the enclosing instance.
- Anonymous class creates its own scope.
- Lambdas are usually more concise and can be optimized better by the JVM.

16) CompletableFuture

Use CompletableFuture for asynchronous, non-blocking composition.

```
CompletableFuture<String> future = CompletableFuture
    .supplyAsync(() -> fetchUser())
    .thenApply(user -> user.toUpperCase())
    .exceptionally(ex -> "DEFAULT");
```

17) Intermediate vs terminal stream operations

- **Intermediate:** lazy, return another stream, e.g. map, filter, sorted.
- **Terminal:** produce result or side effect, e.g. collect, forEach, count.

```
long count = users.stream()
    .filter(u -> u.isActive())
    .map(User::getName)
    .count();
```

18) Parallel streams

Use parallel streams only for CPU-bound, stateless, large-data operations after benchmarking. Avoid them in web apps when request threads are already shared and blocking tasks or custom thread-pool control matter.[file:1]

Pitfalls:

- Uses common ForkJoinPool by default.
- Poor fit for blocking I/O.
- Harder debugging.
- Can reduce performance for small collections.

19) Spliterator vs Iterator

Iterator traverses sequentially. Spliterator can split data for parallel processing and exposes characteristics like SIZED, ORDERED, IMMUTABLE.

20) Multiple inheritance in Java 8

Java does not support multiple inheritance of classes because of ambiguity and state conflicts. It supports multiple inheritance of type through interfaces, and Java 8 added default methods with conflict-resolution rules.

Diamond rule:

- Class method wins over interface default method.
- More specific interface wins.

- If ambiguity remains, overriding is mandatory.